

Name : Vivek Singh
RegNumber : 230905408
Roll Number: A50

Lab 10: Programming using different CUDA device memory types and synchronization

Q1) Write a program in CUDA to perform matrix multiplication using 2D Grid and 2D Block.

Code:

```
// Q1: Matrix Multiplication using 2D Grid and 2D Block
#include <stdio.h>
#include <cuda.h>

#define N 4 // Matrix size NxN

__global__ void matMul(int *A, int *B, int *C, int n) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < n && col < n) {
        int sum = 0;
        for (int k = 0; k < n; k++)
            sum += A[row * n + k] * B[k * n + col];
        C[row * n + col] = sum;
    }
}

int main() {
    int n = N;
    int size = n * n * sizeof(int);
    int h_A[N*N], h_B[N*N], h_C[N*N];

    printf("Matrix A:\n");
    for (int i = 0; i < n * n; i++) {
        h_A[i] = i + 1;
        if ((i + 1) % n == 0) printf("%d\n", h_A[i]);
        else printf("%d ", h_A[i]);
    }
    printf("Matrix B (Identity):\n");
    for (int i = 0; i < n * n; i++) {
        h_B[i] = (i % n == i / n) ? 1 : 0;
        if ((i + 1) % n == 0) printf("%d\n", h_B[i]);
        else printf("%d ", h_B[i]);
    }

    int *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, size);
    cudaMalloc(&d_B, size);
    cudaMalloc(&d_C, size);
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    dim3 blockDim(2, 2);
    dim3 gridDim((n + blockDim.x - 1) / blockDim.x,
                  (n + blockDim.y - 1) / blockDim.y);

    matMul<<<gridDim, blockDim>>>(d_A, d_B, d_C, n);
    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

    printf("Result Matrix C = A x B:\n");
    for (int i = 0; i < n * n; i++) {
        if ((i + 1) % n == 0) printf("%d\n", h_C[i]);
        else printf("%d ", h_C[i]);
    }
}
```

```

}

cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
return 0;
}

```

Output:

```

Matrix A:
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
Matrix B (Identity):
1 0 0 0
0 1 0 0
0 0 1 0
0 0 0 1
Result Matrix C = A x B:
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16

```

Q2) Write a program in CUDA to improve the performance of 1D parallel convolution using constant Memory.

Code:

```

// Q2: 1D Parallel Convolution using Constant Memory
#include <stdio.h>
#include <cuda.h>

#define MASK_SIZE 5
#define ARRAY_SIZE 10

__constant__ int d_M[MASK_SIZE];

__global__ void convolution1D(int *N, int *P, int width, int maskSize) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int halfMask = maskSize / 2;
    if (i < width) {
        int sum = 0;
        for (int j = 0; j < maskSize; j++) {
            int idx = i - halfMask + j;
            if (idx >= 0 && idx < width)
                sum += N[idx] * d_M[j];
        }
        P[i] = sum;
    }
}

int main() {
    int width = ARRAY_SIZE;
    int maskSize = MASK_SIZE;
    int h_N[ARRAY_SIZE] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int h_M[MASK_SIZE] = {1, 2, 3, 2, 1};
    int h_P[ARRAY_SIZE];

    printf("Input Array N: ");
    for (int i = 0; i < width; i++) printf("%d ", h_N[i]);
    printf("\n");
    printf("Mask M: ");

```

```

for (int i = 0; i < maskSize; i++) printf("%d ", h_M[i]);
printf("\n");

cudaMemcpyToSymbol(d_M, h_M, maskSize * sizeof(int));

int *d_N, *d_P;
cudaMalloc(&d_N, width * sizeof(int));
cudaMalloc(&d_P, width * sizeof(int));
cudaMemcpy(d_N, h_N, width * sizeof(int), cudaMemcpyHostToDevice);

int blockSize = 4;
int gridSize = (width + blockSize - 1) / blockSize;
convolution1D<<<gridSize, blockSize>>>(d_N, d_P, width, maskSize);

cudaMemcpy(h_P, d_P, width * sizeof(int), cudaMemcpyDeviceToHost);
printf("Output Array P: ");
for (int i = 0; i < width; i++) printf("%d ", h_P[i]);
printf("\n");

cudaFree(d_N); cudaFree(d_P);
return 0;
}

```

Output:

Input Array N: 1 2 3 4 5 6 7 8 9 10
 Mask M: 1 2 3 2 1
 Output Array P: 8 14 22 30 38 46 54 62 60 47

Q3) Write a program in CUDA to perform tiled 1D convolution operation on the input array N of size width using the mask array M of size mask_width, to produce the resultant array P of size width.

Code:

```

// Q3: Tiled 1D Convolution using Shared Memory
#include <stdio.h>
#include <cuda.h>

#define MASK_SIZE 5
#define TILE_SIZE 8
#define WIDTH 16

__constant__ int d_M[MASK_SIZE];

__global__ void tiledConvolution1D(int *N, int *P, int width, int maskSize) {
    int halfMask = maskSize / 2;
    int tx = threadIdx.x;
    int i = blockIdx.x * TILE_SIZE + tx;

    __shared__ int N_s[TILE_SIZE + MASK_SIZE - 1];

    int haloLeft = i - halfMask;
    if (tx < halfMask)
        N_s[tx] = (haloLeft >= 0) ? N[haloLeft] : 0;

    if (i < width) N_s[tx + halfMask] = N[i];
    else          N_s[tx + halfMask] = 0;

    if (tx >= TILE_SIZE - halfMask) {
        int haloRight = i + halfMask;
        N_s[tx + 2 * halfMask] = (haloRight < width) ? N[haloRight] : 0;
    }
}

```

```

}

__syncthreads();

if (i < width) {
    int sum = 0;
    for (int j = 0; j < maskSize; j++)
        sum += N_s[tx + j] * d_M[j];
    P[i] = sum;
}
}

int main() {
    int width = WIDTH;
    int maskSize = MASK_SIZE;
    int h_N[WIDTH] = {1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16};
    int h_M[MASK_SIZE] = {1, 2, 3, 2, 1};
    int h_P[WIDTH];

    printf("Input Array N (size %d):\n", width);
    for (int i = 0; i < width; i++) printf("%d ", h_N[i]);
    printf("\n");
    printf("Mask M (size %d): ", maskSize);
    for (int i = 0; i < maskSize; i++) printf("%d ", h_M[i]);
    printf("\n");

    cudaMemcpyToSymbol(d_M, h_M, maskSize * sizeof(int));

    int *d_N, *d_P;
    cudaMalloc(&d_N, width * sizeof(int));
    cudaMalloc(&d_P, width * sizeof(int));
    cudaMemcpy(d_N, h_N, width * sizeof(int), cudaMemcpyHostToDevice);

    int gridSize = (width + TILE_SIZE - 1) / TILE_SIZE;
    tiledConvolution1D<<<gridSize, TILE_SIZE>>>(d_N, d_P, width, maskSize);

    cudaMemcpy(h_P, d_P, width * sizeof(int), cudaMemcpyDeviceToHost);
    printf("Output Array P:\n");
    for (int i = 0; i < width; i++) printf("%d ", h_P[i]);
    printf("\n");

    cudaFree(d_N); cudaFree(d_P);
    return 0;
}

```

Output:

```

Input Array N (size 16):
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
Mask M (size 5): 1 2 3 2 1
Output Array P:
8 14 22 30 38 46 54 62 70 78 86 94 102 110 108 95

```